NoSQL Databases

Francesco Pelacani
2025-2026

Introduction

practice

How would you design Instagram?

The value of relational Databases

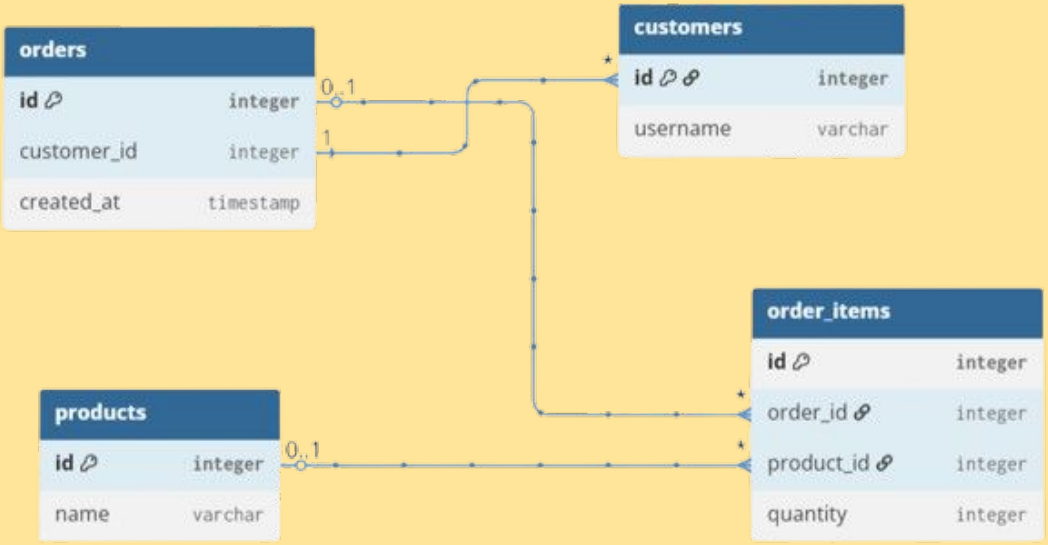
Relational databases have been the default choice for serious data storage for decades.

Their benefits:

- **Getting at Persistent Data**
- **Concurrency**
- **Integration**
- **A (mostly) standard model**

practice

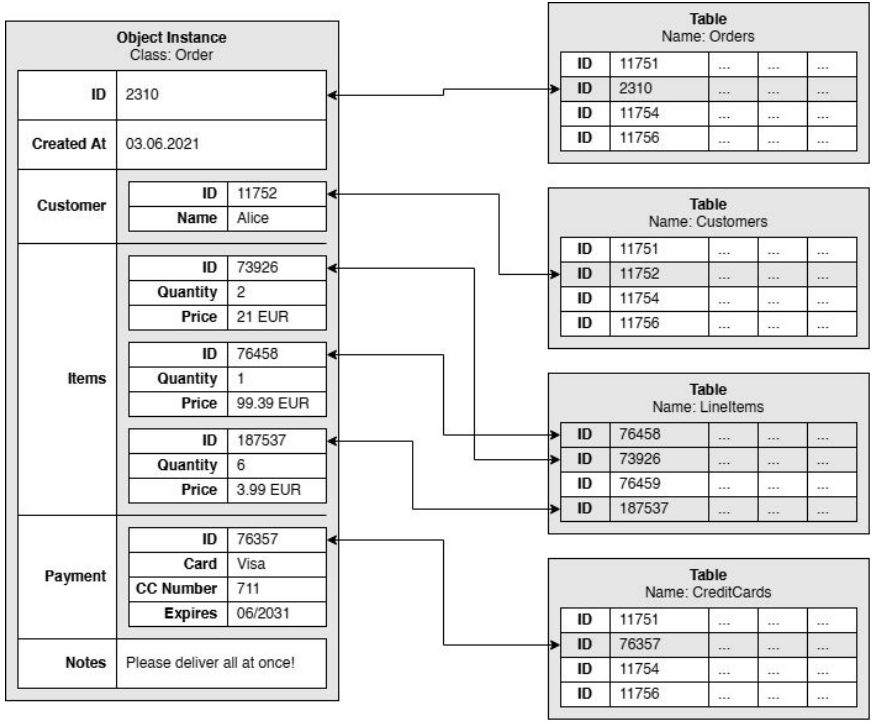
SQL in short



Impedance mismatch

Is the difference between the relational model and the in-memory data structures.

Relational data model organizes data into structure of **rows** (*tuples*, a set of name-value pairs) and **tables** (*relations*, a set of tuples).



practice

Object-Relational Impedance Mismatch

Java

```
customer.getOrders().get(0).getProduct().getName();
```

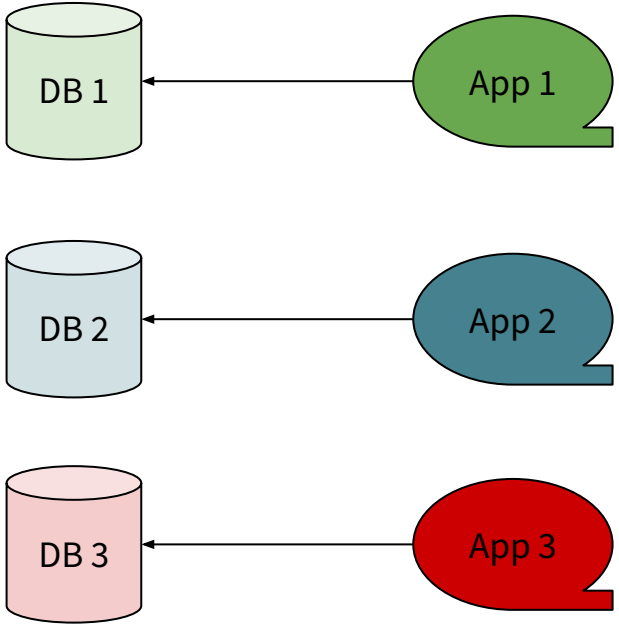
SQL

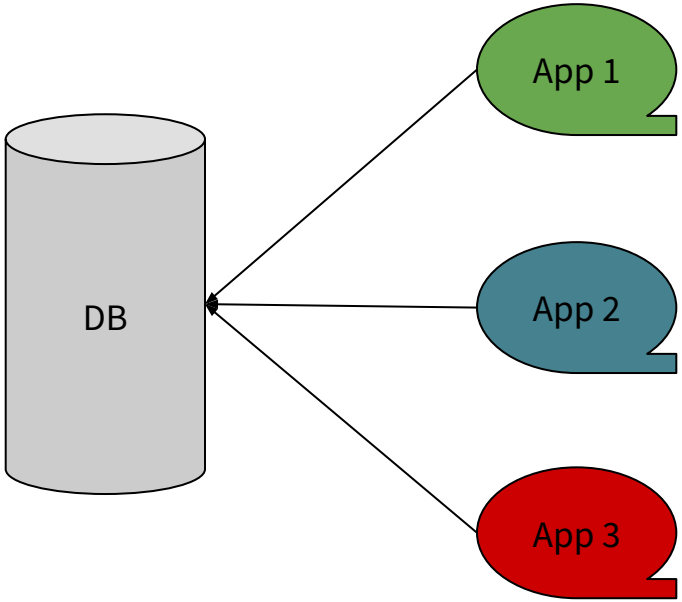
```
SELECT p.name FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN order items oi ON o.id = oi.order id
JOIN products p ON oi.product_id = p.id
WHERE c.id = 123;
```

The 6 dimensions of Impedance Mismatch

- **Granularity** → Fine-Grained Objects vs Flat Tables
- **Inheritance** → Is-A Relationships Meet SQL
- **Identity** → The “Identity Crisis” (object equality vs primary keys)
- **Associations** → Pointers vs Foreign Keys
- **Navigation** → Graph Walking vs Set-Based Queries
- **Encapsulation / Behavior** → Tell, Don’t Ask Meets CRUD

Application and Integration Databases





Attack of the Clusters

In the early 2000s large sets of data started to appear, along many dimensions.

To handle this increase, you have 2 options: **up** or **out**.

Up: bigger machines, more processors, disk storage, and memory → High costs, hard limits.

Out: run small machines in a cluster → cheaper and more resilient.

Relational databases are not designed to be run on clusters.

Google and Amazon, started to cook something. Respectively BigTable and Dynamo.

The Emergence of NoSQL

NoSQL is an accidental neologism, the common characteristics of NoSQL databases are:

- **Not using the relational model**
- **Running well on clusters**
- **Open-source**
- **Schemaless**

The most important result of the rise of NoSQL is **Polyglot Persistence**: using different data stores in different circumstances.

practice

Kahoot #1

Aggregate Data Models

What is an Aggregate?

A collection of related objects that we wish to treat as a unit.

Typically we want **to update aggregates atomically** and **to communicate with our data storage in terms of aggregates**.

Aggregate-oriented databases work best when most data interaction is done with the same aggregate; aggregate-ignorant databases are better when interactions use data organized in many different formations

practice

An example of Relations and Aggregates

Imagine you are building an e-commerce website. You need to store data for selling items to customers, which includes: users, the product catalog, orders, shipping addresses, billing addresses, and payment data.

practice

Part 1: The Relational Approach

Your task: Think about how you would break this data down into a traditional Entity-Relationship (E/R) model using normalized tables.

Learning Notes: In a relational database, data is strictly organized into tables (relations) and rows (tuples). To keep data consistent and avoid duplication, you must split the information across multiple tables and link them using IDs (foreign keys).

practice

Part 2: The NoSQL Document Approach

Your Task: Now, rethink that exact same order, but design it as a single *Document* "Aggregate".

Learning Notes: NoSQL Document databases use an "Aggregate Data Model". An aggregate is simply a collection of related objects that you treat as one single unit. Document models are schemaless, meaning you don't have to define rigid tables beforehand. Instead of splitting the order up, you nest the related information together (often in a format like JSON).

Aggregate-Oriented Databases

Key-value data model → the aggregate is opaque to the database, lookup by key for the whole aggregate

Document model → the aggregate is transparent to the database allowing queries and partial retrievals. Schemaless: no storage nor retrieve optimization.

Column-family model → the aggregate is divided into column families, allowing the database to treat them as units of data within row aggregate. This imposes some structure but improves accessibility.

More details on Data Models

Relationships and Graph DBs

Aggregate-oriented databases make inter-aggregate relationships more difficult to handle than intra-aggregate relationships.

Graph Databases specialize in capturing complex relationships, by organizing data into node and edge graphs. Traversal along the relationships is very cheap, because the navigating costs are paid up-front at insert time (and not at query time like relational databases joins).

Schemaless Databases

NoSQL databases does not need a schema definition in advance, you can easily store whatever you need (even **nonuniform data**).

However there is usually an implicit schema present, a set of assumptions about the data's structure in the code that manipulates the data.

Materialized views

Relational databases have a mechanism to supporto data accessing in different ways: **materialized views**. Views computed in advance and cached on disk.

NoSQL databases doesn’t have the concept of views, they may have precomputed and cached queries. They can be computed at insert time or at regular intervals by batch jobs, it depends on the requirements about data access.

Distribution Models

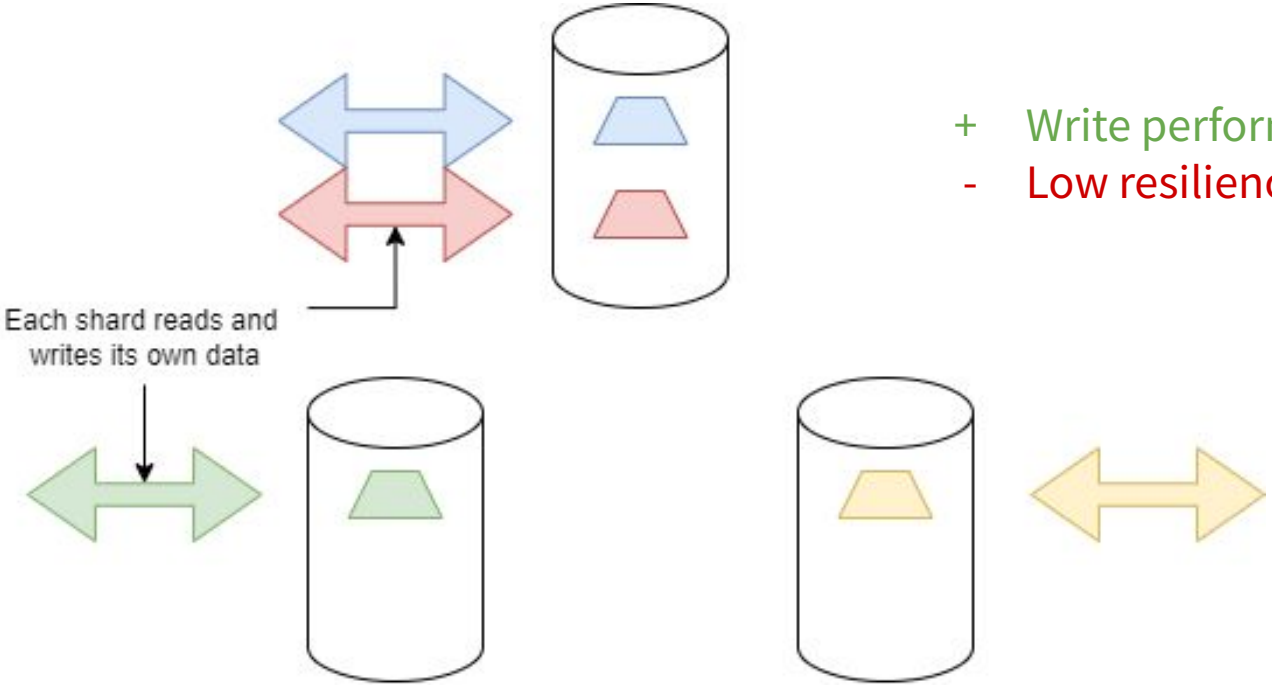
Single Server

No distribution at all.
The database runs on a single machine that handles all the reads and writes to the data store.

Graph databases work best in a single server configuration.

Sharding

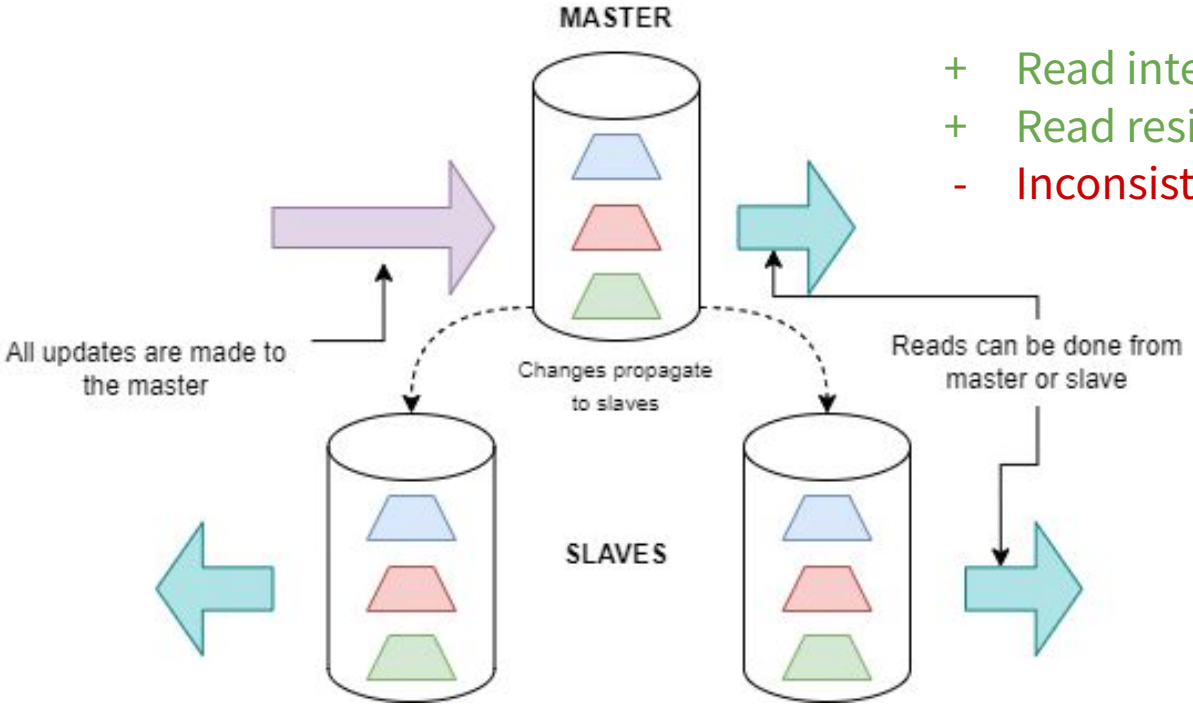
Put different pars of the data onto different servers.



+ Write performance
- Low resilience

Master-Slave Replication

Put different pars of the data onto different servers.

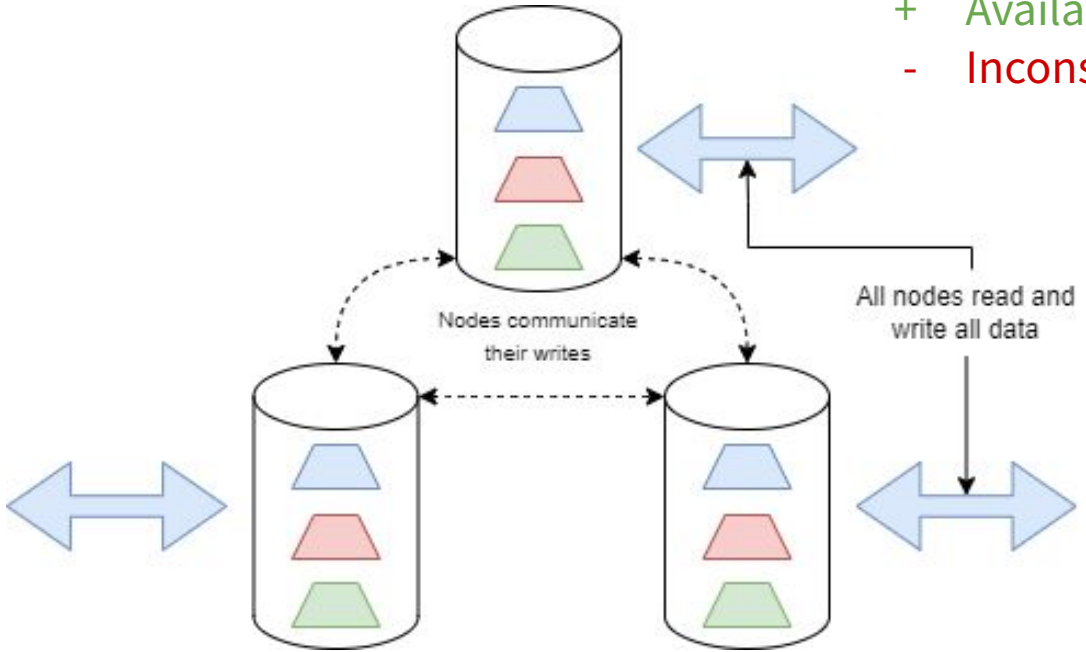


+ Read intensive dataset
+ Read resilience
- Inconsistency

Peer-to-Peer Replication

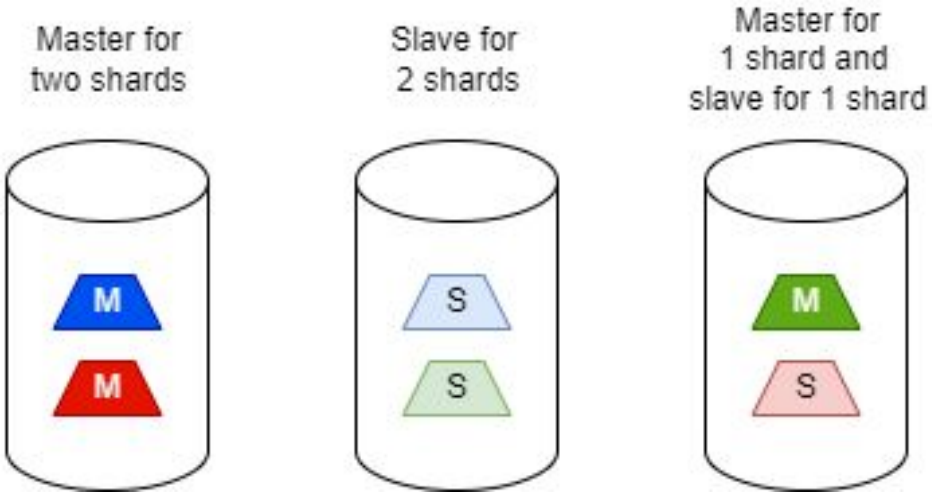
All replicas have equal weight, they can all accept writes, and the loss of any of them doesn't prevent access to the data.

+ Availability
- Inconsistency



Combining Sharding and Replication

If we use both master-slave replication and sharding, we ends up having multiple masters, but each data item only has a single master.



Consistency

Shapes of Consistency

Update consistency: two people updating the same data item at the same time (write-write conflicts).

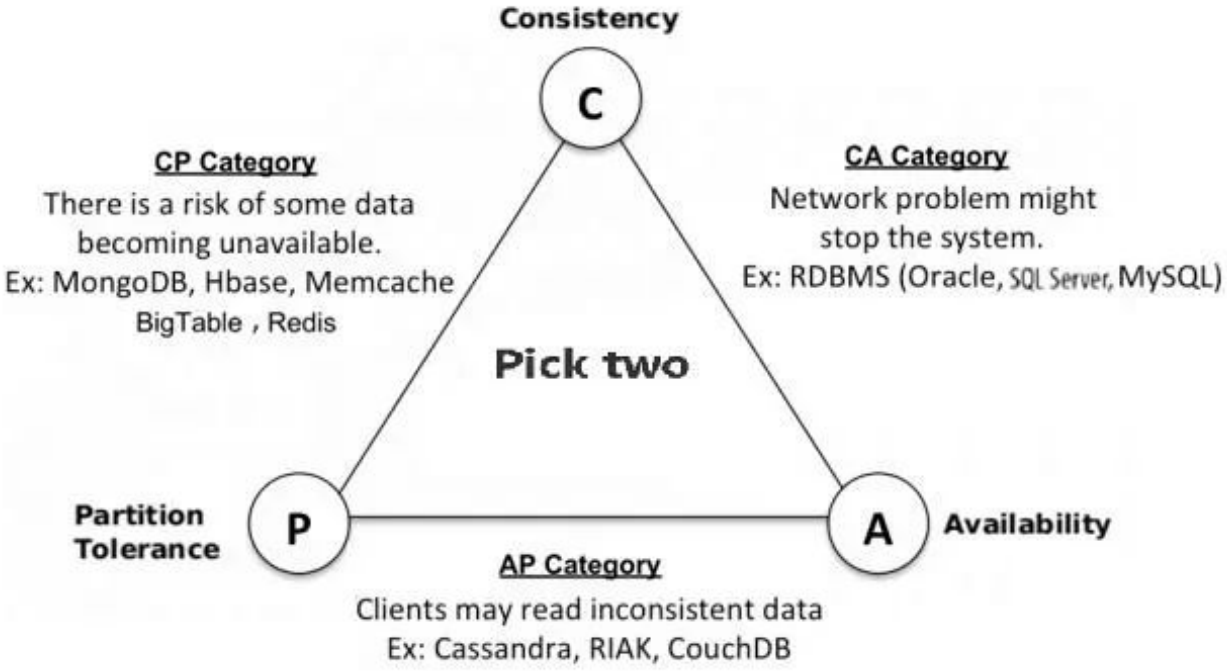
Read consistency: one client reads inconsistent data in the middle of another client's write (read-write conflicts). Distributed systems might have nodes that have received updates while other nodes have not (eventual consistency).

How to maintain Consistency?

- **Pessimistic approach** → prevents conflicts from occurring. E.g. write locks, to change a value you need to acquire a lock, and the system ensures that only one client can get a lock at a time
- **Optimistic approach** → lets conflicts occur, but detects them and takes action to sort them out. E.g. conditional update, before pushing an update a client test the value to see if it's changed since his last read (*version stamps*).

The CAP Theorem

If you get a network partition, you have to trade off availability of data versus consistency.



Relaxing durability

What is the point of a data store if it can lose updates?

Performance

Especially during replication you might not need to contact all replicants to preserve strong consistency; you just need a large enough quorum.

Quorums

How many nodes needs to be involved to get strong consistency?

The relationship between the number of nodes you need to contact for a read (**R**), those confirming a write (**W**), and the replication factor (**N**) can be captured by this inequality:

You get strongly consistent read if **R + W > N**.

What if … ?

practice

Version Stamps

What Are Version Stamps in NoSQL?

Version stamps are special **metadata values** attached to a record to indicate its exact version at a specific point in time.

They are commonly used in NoSQL databases to:

- Track changes to data
- Prevent write conflicts
- Enable optimistic concurrency control
- Support ordering of updates in distributed systems

Think of a version stamp as a unique "change identifier" for a record.

Why Version Stamps Are Needed

NoSQL databases are often:

- Distributed across multiple nodes
- Eventually consistent
- Designed for high concurrency

This creates challenges:

- Two clients may update the same record simultaneously
- Updates may arrive out of order
- Replicas may temporarily disagree

Version stamps help detect and resolve these conflicts safely.

How Version Stamps Work

Typical workflow:

1. A record is stored with a version stamp

   { id: 101, name: "Alice", version: 5 }

2. Client reads the record (version = 5)
3. Client attempts update:

   Update where id=101 AND version=5 set name="Alice Smith" increment version to 6

4. If another update already changed version to 6:
   a. The update fails
   b. Client must retry

This is called **Optimistic Concurrency Control** (**OCC**).

What Does a Version Stamp Look Like?

Depending on the database, it can be:

- A simple integer (1, 2, 3…)
- A timestamp
- A logical clock value
- A hybrid logical timestamp
- A globally ordered transaction ID

Map-Reduce

What Is MapReduce?

MapReduce is a distributed programming model used to **process large-scale data across clusters of machines**.

It is designed for:

- Massive datasets (TBs–PBs)
- Distributed storage systems
- Parallel computation
- Fault-tolerant processing

It became popular through *Google* and later open-sourced in ecosystems like *Apache Hadoop*.

Why MapReduce in NoSQL Systems?

NoSQL databases and distributed storage systems:

- Store huge volumes of data
- Distribute data across many nodes
- Prioritize scalability over strict relational models

Traditional SQL engines don’t always scale horizontally well.

MapReduce enables data-local, parallel computation, meaning:

- Computation runs where data lives
- Network transfer is minimized
- Work is distributed automatically

The Two Core Phases (1)

Map Phase

- Input data is split into chunks
- Each node processes its chunk independently
- Outputs key–value pairs

Input: "cat dog cat"
Map Output:
(cat,1)
(dog,1)
(cat,1)

The Two Core Phases (2)

System groups values by key

Reducers aggregate results.

(cat, [1,1]) → (cat,2)
(dog, [1]) → (dog,1)

Final output:

cat: 2
dog: 1

Partitioning (How Data Is Distributed to Reducers)

After the Map phase, the system must decide which reducer processes which key?

This is done via a **partitioner** -> Each key is passed to a partition function (Common strategy: *hash(key) % number_of_reducers*)

This ensures:

- Same keys always go to the same reducer
- Load is (ideally) evenly distributed

Partitioning is critical for:

- Parallelism
- Load balancing
- Scalability

Combining (Local Pre-Aggregation Optimization)

A combiner is an optional mini-reducer that runs after the Map phase and before the Shuffle phase

Its purpose:

- Reduce data sent over the network
- Perform local aggregation

Why This Matters

- The shuffle phase is network-heavy.
- Reduce bandwidth usage
- Reduce disk I/O
- Improve performance significantly

Combiners only work for operations that are *associative* and *commutative* (e.g., sum, count, min, max)